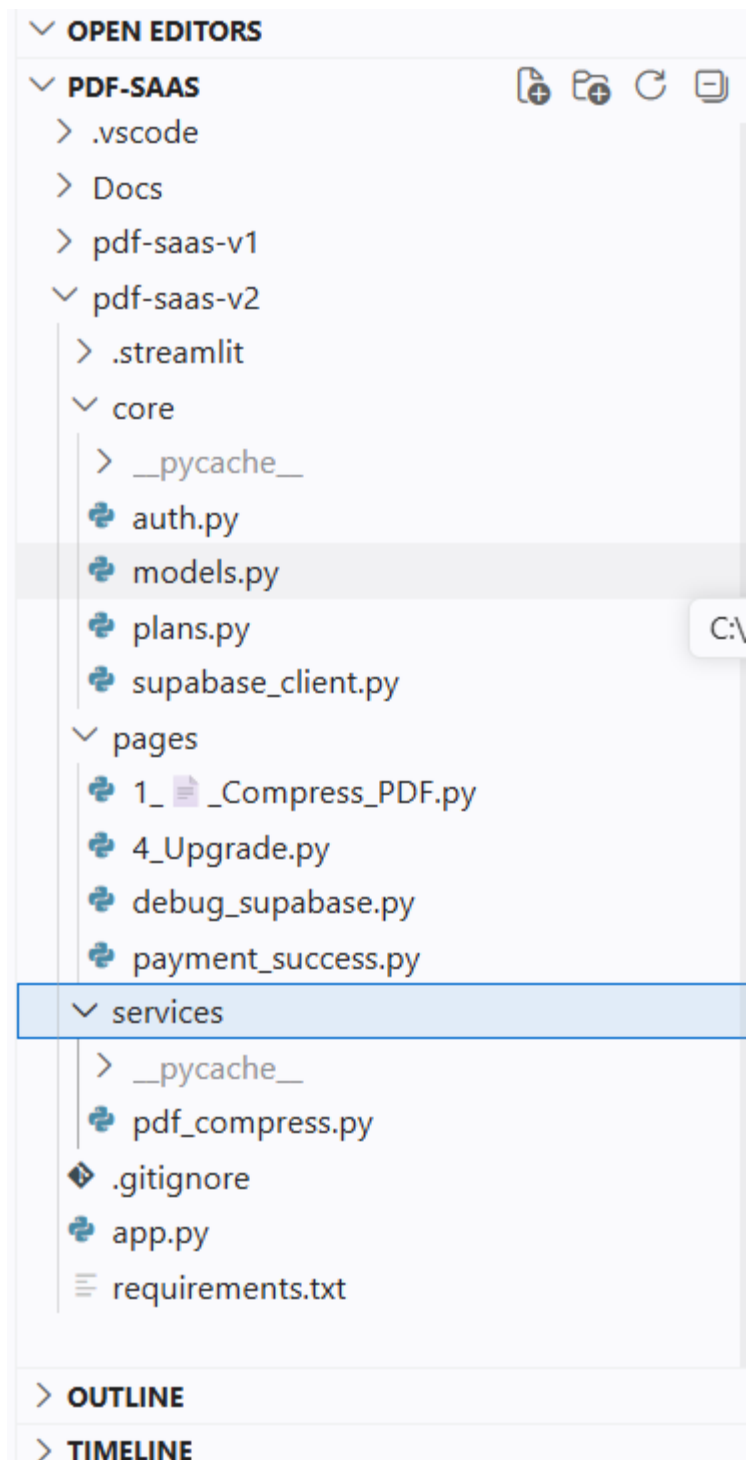




auth.py code:

```
import streamlit as st
```

```
from .supabase_client import get_supabase
```

```
from .models import User
```

```
SESSION_USER_KEY = "user"
```

```
PAGE_CONFIG_FLAG = "page_config_set"
```

```
def _set_page_config():
```

```
    if PAGE_CONFIG_FLAG not in st.session_state:
```

```
        st.set_page_config(page_title="PDF SaaS", page_icon="📄", layout="wide")
```

```
        st.session_state[PAGE_CONFIG_FLAG] = True
```

```
def _init_session():
```

```
    if SESSION_USER_KEY not in st.session_state:
```

```
        st.session_state[SESSION_USER_KEY] = None
```

```
def get_current_user() -> User | None:
```

```
    return st.session_state.get(SESSION_USER_KEY)
```

```
def _store_user(res_user):
```

```
    st.session_state[SESSION_USER_KEY] = User(
```

```
        id=res_user.id,
```

```
        email=res_user.email,
```

```
    )
```

```
def login_form():
```

```
    sb = get_supabase()
```

```
    st.title("Login to PDF SaaS")
```

```

with st.form("login_form"):
    email = st.text_input("Email")
    password = st.text_input("Password", type="password")
    submitted = st.form_submit_button("Login")

if not submitted:
    return

if not email or not password:
    st.error("Please enter both email and password.")
    return

try:
    res = sb.auth.sign_in_with_password({"email": email, "password": password})
    if res.user:
        _store_user(res.user)
        st.success("Logged in successfully.")
        st.rerun()
    else:
        st.error("Login failed. Check credentials.")
except Exception as e:
    msg = getattr(e, "message", str(e))
    st.error("Login error.")
    st.caption(msg)

```

```
def logout():
    sb = get_supabase()
    try:
        sb.auth.sign_out()
    except Exception:
        pass

    st.session_state[SESSION_USER_KEY] = None
    st.success("Logged out.")
    st.rerun()
```

```
def require_auth() -> User:
```

```
    _set_page_config()
```

```
    _init_session()
```

```
    user = get_current_user()
```

```
    if user is None:
```

```
        login_form()
```

```
        st.stop()
```

```
    return user
```

models.py code:

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class User:
```

id: str

email: str

@dataclass

class Plan:

id: str

slug: str

name: str

price_monthly: int

max_file_size_mb: int

max_files_per_merge: int

ocr_enabled: bool

plans.py code:

from .supabase_client import get_supabase

from .models import Plan, User

FREE_PLAN = Plan(

id="free-local",

slug="free",

name="Free",

price_monthly=0,

max_file_size_mb=5,

max_files_per_merge=2,

ocr_enabled=False,

)

```
def _plan_from_row(row) -> Plan:
```

```
    return Plan(
        id=row["id"],
        slug=row["slug"],
        name=row["name"],
        price_monthly=row["price_monthly"],
        max_file_size_mb=row["max_file_size_mb"],
        max_files_per_merge=row["max_files_per_merge"],
        ocr_enabled=row["ocr_enabled"],
    )
```

```
def get_active_subscription_plan(user: User) -> Plan:
```

```
    sb = get_supabase()
    res = (
        sb.table("subscriptions")
        .select("status, current_period_end, plans(*)")
        .eq("user_id", user.id)
        .in_("status", ["trialing", "active"])
        .order("current_period_end", desc=True)
        .limit(1)
        .execute()
    )
```

```
    data = res.data or []
```

```
    if not data or not data[0].get("plans"):
```

```
return FREE_PLAN
```

```
return _plan_from_row(data[0]["plans"])
```

```
def enforce_limits(plan: Plan, action: str, file_count=1, file_size_mb=0.0) -> None:
```

```
    if file_size_mb > plan.max_file_size_mb:
```

```
        raise ValueError(f"File too large for your plan (max {plan.max_file_size_mb} MB).")
```

```
    if action == "merge" and file_count > plan.max_files_per_merge:
```

```
        raise ValueError(f"Too many files for your plan (max {plan.max_files_per_merge}).")
```

```
    if action == "ocr" and not plan.ocr_enabled:
```

```
        raise ValueError("OCR is only available on higher plans.")
```

supabase_client.py code:

```
import streamlit as st
```

```
from supabase import create_client
```

```
_supabase = None
```

```
def get_supabase():
```

```
    global _supabase
```

```
    if _supabase is None:
```

```
        url = st.secrets["supabase"]["url"]
```

```
        key = st.secrets["supabase"]["key"]
```

```
        _supabase = create_client(url, key)
```

```
return _supabase
```

under pages

1_  _Compress_PDF.py code:

```
import streamlit as st
```

```
from core.auth import require_auth
```

```
from core.plans import get_active_subscription_plan, enforce_limits
```

```
from services.pdf_compress import compress_pdf
```

```
st.title("Compress PDF")
```

```
user = require_auth()
```

```
plan = get_active_subscription_plan(user)
```

```
uploaded = st.file_uploader("Upload PDF", type=["pdf"])
```

```
if uploaded:
```

```
    pdf_bytes = uploaded.read()
```

```
    size_mb = len(pdf_bytes) / (1024 * 1024)
```

```
    st.info(f"Uploaded file size: {size_mb:.2f} MB")
```

```
    st.caption(f"Plan: {plan.name} (max {plan.max_file_size_mb} MB)")
```

```
    try:
```

```
        enforce_limits(plan, action="compress", file_size_mb=size_mb)
```

```
    except ValueError as e:
```



```
st.error(str(e))  
  
st.warning("Upgrade your plan to compress larger files.")  
  
st.page_link("pages/4_Upgrade.py", label="Go to Upgrade Page")  
  
st.stop()
```

```
if st.button("Compress"):  
    with st.spinner("Compressing..."):  
        out_bytes = compress_pdf(pdf_bytes)  
  
    st.success("Compression complete.")  
  
    st.download_button(  
        "Download compressed PDF",  
        out_bytes,  
        "compressed.pdf",  
        mime="application/pdf",  
    )
```

4_Upgrade.py code :

```
import streamlit as st  
  
from core.auth import require_auth  
from core.supabase_client import get_supabase  
from core.models import User  
  
st.title("Upgrade Your Plan")  
  
user: User = require_auth()
```

```
sb = get_supabase()
```

```
# -----
```

```
# SAFE Supabase query for plans
```

```
# -----
```

```
res = (
```

```
    sb.table("plans")
```

```
    .select("*")
```

```
    .order("price_monthly")
```

```
    .execute()
```

```
)
```

```
if not res or not hasattr(res, "data") or res.data is None:
```

```
    st.error("Could not load plans. Check Supabase or RLS policies.")
```

```
    st.stop()
```

```
plans = res.data
```

```
# -----
```

```
# Fetch current subscription
```

```
# -----
```

```
sub_res = (
```

```
    sb.table("subscriptions")
```

```
    .select("*")
```

```
    .eq("user_id", user.id)
```

```
    .eq("status", "active")
```

```
.maybe_single()
.execute()
)
```

```
sub = sub_res.data if sub_res and hasattr(sub_res, "data") else None
current_plan_id = sub["plan_id"] if sub else None
```

```
st.write("Choose a plan to unlock higher limits and premium features.")
```

```
cols = st.columns(len(plans))
```

```
for idx, plan in enumerate(plans):
```

```
    with cols[idx]:
```

```
        st.subheader(plan["name"])

```

```
        st.write(f"***₹{plan['price_monthly']} / month***")

```

```
        st.write(f"Max File Size: **{plan['max_file_size_mb']} MB**")

```

```
        st.write(f"Max Merge Files: **{plan['max_files_per_merge']}**")

```

```
        st.write(f"OCR Enabled: **{'Yes' if plan['ocr_enabled'] else 'No'}**")

```

```
    if plan["id"] == current_plan_id:
```

```
        st.success("Your Current Plan")

```

```
        continue

```

```
    if st.button(f"Upgrade to {plan['name']}", key=plan["id"]):
```

```
        # Update existing subscription
```

```

if sub:

    sb.table("subscriptions").update({

        "plan_id": plan["id"],

        "status": "active"

    }).eq("id", sub["id"]).execute()

# Create new subscription
else:

    sb.table("subscriptions").insert({

        "user_id": user.id,

        "plan_id": plan["id"],

        "status": "active"

    }).execute()

st.success(f"Successfully upgraded to {plan['name']}!")
st.rerun()

```

payment_success.py code :

```

import streamlit as st

from core.auth import require_auth

from core.supabase_client import get_supabase

```

```

st.title("Payment Successful")

```

```

user = require_auth()

```

```

sb = get_supabase()

```

```
order_id = st.session_state.get("order_id")
plan_id = st.session_state.get("selected_plan_id")
```

```
if not order_id or not plan_id:
    st.error("Invalid payment session.")
    st.stop()
```

```
# Fetch existing subscription
```

```
sub = (
    sb.table("subscriptions")
    .select("*")
    .eq("user_id", user.id)
    .maybe_single()
    .execute()
    .data
)
```

```
# Update or create subscription
```

```
if sub:
    sb.table("subscriptions").update({
        "plan_id": plan_id,
        "status": "active"
    }).eq("id", sub["id"]).execute()
```

```
else:
```

```
    sb.table("subscriptions").insert({
```

```

        "user_id": user.id,
        "plan_id": plan_id,
        "status": "active"
    }).execute()

st.success("Your plan has been upgraded successfully!")

st.balloons()

st.page_link("app.py", label="Go to Dashboard")

under services

pdf_compress.py code:

from io import BytesIO
import pikepdf

def compress_pdf(pdf_bytes: bytes) -> bytes:
    input_pdf = BytesIO(pdf_bytes)
    output_pdf = BytesIO()

    with pikepdf.open(input_pdf) as pdf:
        pdf.save(
            output_pdf,
            compress_streams=True,
            object_stream_mode=pikepdf.ObjectStreamMode.generate,
            linearize=True,
        )

```

```
return output_pdf.getvalue()
```

.gitignore code:

```
# -----
```

```
# Python
```

```
# -----
```

```
__pycache__/
```

```
*.py[cod]
```

```
*.pyo
```

```
*.pyd
```

```
*.pdb
```

```
*.egg-info/
```

```
*.egg
```

```
*.log
```

```
# Virtual environments
```

```
.venv/
```

```
.supabase-venv/
```

```
venv/
```

```
env/
```

```
ENV/
```

```
# Jupyter
```

```
.ipynb_checkpoints/
```

Streamlit

.streamlit/secrets.toml

.streamlit/config.toml

.streamlit/state.json

Streamlit cache

**/.streamlit/cache/

**/.streamlit/cache/*

Supabase

supabase/.temp/

supabase/.branches/

supabase/.config/

PDF Processing Temp Folders

uploads/

output/

processed/

temp/

tmp/

cache/

ocr_temp/

Large sample files

*.pdf

!assets/sample_pdfs/*

OS Junk

.DS_Store

Thumbs.db

desktop.ini

IDE / Editor

.vscode/

.vscode/*

.idea/

*.code-workspace

Environment / Secrets

.env

.env.*

*.env

secrets.json

Build / Distribution

build/

dist/

*.spec

Logs

*.log

logs/

app.py code:

import streamlit as st

from core.auth import require_auth, logout

from core.plans import get_active_subscription_plan

user = require_auth()

plan = get_active_subscription_plan(user)

```
st.sidebar.title("PDF SaaS")  
st.sidebar.write(user.email)  
st.sidebar.write(f"Plan: {plan.name}")  
if st.sidebar.button("Logout"):  
    logout()
```

```
st.title("Welcome to PDF SaaS")  
st.write("Choose a tool from the sidebar pages:")  
st.write("- Compress PDF")  
st.write("- Merge PDFs")  
st.write("- OCR Reader")
```

requirements.txt code:

```
streamlit  
supabase  
pikepdf  
python-dotenv  
razorpay
```